# Customer Golden Record — Design & Decision Note

## Purpose

Resolve likely duplicate customer records from two synthetic source systems, standardize fields, create a traceable golden record, and make each match decision readable to a human or downstream AI workflow.

## Approach

A Python pipeline normalizes whitespace, email case, phone digits, names, cities, and postal codes. It compares CRM records with billing records using deterministic rules: exact email, exact phone, then fuzzy name similarity corroborated by postal code. Similar names with weaker location evidence are flagged for review rather than merged.

## Decisions and trade-offs

• Use only the Python standard library and SQLite for a clean, low-friction setup.
• Use a deterministic matcher so results are reproducible and auditable.
• Keep explanation generation behind an interface: rules are the default; optional local Ollama only verbalizes supplied evidence and never decides the match.
• Merge only `match` decisions; keep `review` pairs separate to reduce false positives.
• Choose non-empty golden fields using CRM-first source priority and retain source references.

## Validation

Automated tests cover exact normalized email, fuzzy matching with postal-code corroboration, a same-name/different-location edge case, transitive clustering behavior, and empty inputs. Outputs include CSV/JSON/JSONL plus a SQLite audit model.

## Known limitations

Pairwise comparison is O(N × M). Similarity thresholds are illustrative and need calibration on labeled data. Connected components can propagate a false link; a production version should add cluster contradiction checks, blocking for scale, provenance/recency-based field survivorship, and a human review queue.

## Run

`python -m pip install -e .` then `customer-golden-record`; tests: `python -m unittest discover -s tests -v`.